

Problem Statement 01: Predictive Maintenance

Background

Naval and merchant ships depend on continuous propulsion and auxiliary machinery across harsh maritime environments. Unplanned mechanical breakdowns at sea compromise operational readiness, risk vessel safety, and inflict heavy maintenance and dockyard expenses.

Problem

Marine machinery operates under fluctuating engine loads (harbour idle, cruise, full power), masking early degradation patterns. Sensor streams frequently contain transient dropouts, noise, and missing entries. Maintenance crews need advance warning of both the time remaining before failure and the exact failing component.

Aim

Develop a machine learning model that ingests multi-sensor time-series logs (temperatures, pressures, vibrations, fuel rack metrics) to accurately predict the Remaining Useful Life (RUL in hours) and classify the specific failure mode (failure vs. no failure).

Output

1. Complete, reproducible training and inference source code (train.py, predict.py)
2. Exported model weights/checkpoints (.pt or .onnx)
3. Predictions CSV mapping unit_id to predicted failure probability and RUL

Data

File Structure:

PS1_public/

├── train.csv

├── test.csv

└── sample_submission.csv

Data Description:

Column	Description	Type
unit_id	Machine identifier	int
timestamp	Sensor reading timestamp	datetime
temperature_c	Operating temperature (°C)	float
pressure_kpa	System pressure (kPa)	float
vibration_hz	Vibration frequency (Hz)	float
motor_current_a	Motor current (Amperes)	float
rotational_speed_rpm	Rotational speed (RPM)	int
torque_nm	Torque (Newton-meters)	float
target_failure	Will it fail? (0/1)	int
rul_hours	Hours until failure	float

Note: target_failure and rul_hours are only in train.csv

Submission Format

File: teamname_PS1.csv

1. unit_id,predicted_failure_prob,predicted_rul_hours
2. 1,0.85,120.5
3. 2,0.12,450.2
4. 3,0.67,89.3

Requirements:

1. Must have exactly 3 columns: unit_id, predicted_failure_prob, predicted_rul_hours
2. predicted_failure_prob: Float between 0.0 and 1.0
3. predicted_rul_hours: Float ≥ 0.0
4. No missing values (NaN)
5. Same number of rows as test.csv
6. File must be named: teamname_PS1.csv

Evaluation Metric

Your submission will be scored on:

1. **AUC-ROC** (classification): Higher is better (70% weight)
2. **RMSE** (regression): Lower is better (30% weight)

Final Score = AUC - (RMSE / 1000) (Higher score wins)

Problem Statement 02: Oil Spill Detection

Background

Major maritime oil tanker casualties release large volumes of heavy crude, creating environmental disasters that threaten vulnerable shorelines, marine sanctuaries, port infrastructure, and coastal desalination intakes.

Problem

Oil slick transport is governed by complex physical dynamics (Fay spreading, surface current advection, wind drift, and evaporation weathering). Spill response commanders have limited physical containment resources (skimmers, booms, dispersant aircraft) and must deploy them under strict environmental exclusion zones prohibiting chemical dispersants near coastal fringes.

Aim

Develop a machine learning model that detects oil spills from GNSS-R Delay-Doppler Map (DDM) images and estimates the oil film thickness.

Output

1. Complete, reproducible training and inference source code (train.py, predict.py)
2. Exported model weights/checkpoints (.pt or .onnx)
3. Predictions CSV mapping sample_id to oil presence probability and thickness

Data

File Structure:

PS2_public/

├── train/images/

| ├── img_001.png

| ├── img_002.png

| └── ...

└── test/images/

```
| |— img_501.png
| |— img_502.png
| |— ...
|— train_labels.csv
|— sample_submission.csv
```

Data Description:

Images:

1. **Format:** PNG (grayscale)
2. **Size:** 400 × 400 pixels
3. **Type:** GNSS-R Delay-Doppler Maps (DDM)
4. **Physical meaning:** Oil spills dampen sea surface roughness, altering GPS signal reflection patterns

train_labels.csv:

Column	Description	Type
sample_id	Unique image identifier	int
image_path	Relative path to image	string
oil_present	Is there oil? (0/1)	int
oil_thickness_mm	Oil thickness (mm)	float

Note: Labels are only in train_labels.csv

Submission Format

File: teamname_PS2.csv

1. sample_id,oil_present_prob,oil_thickness_mm

2. 501,0.95,2.3
3. 502,0.12,0.0
4. 503,0.78,1.5

Requirements:

1. Must have exactly 3 columns: sample_id, oil_present_prob, oil_thickness_mm
2. oil_present_prob: Float between 0.0 and 1.0
3. oil_thickness_mm: Float ≥ 0.0
4. No missing values (NaN)
5. Same number of rows as test set (250 rows)
6. sample_id must match test image filenames
7. File must be named: teamname_PS2.csv

Evaluation Metric

Your submission will be scored on:

1. **LogLoss** (classification): Lower is better (60% weight)
2. **RMSE** (regression): Lower is better (40% weight)

Final Score = LogLoss + (RMSE / 10) (Lower score wins)

Problem Statement 03: Illegal Fishing Detection

Background

Illegal, Unreported, and Unregulated (IUU) fishing fleets exploit vast sovereign Exclusive Economic Zones. These industrial operators often evade detection by turning off AIS transponders, spoofing kinematic vessel profiles, and transferring catches to refrigerated cargo reefers in adjacent international waters.

Problem

Sensor data across the EEZ is asynchronous, noisy, and heterogeneous: AIS feeds contain deliberate dropouts, satellite SAR passes occur at discrete intervals under cloud cover, and coastal radar is constrained by line-of-sight. Enforcement agencies must accurately differentiate illicit industrial actors from legitimate domestic fishers while operating under strict patrol fuel budgets.

Aim

Develop a multi-sensor data fusion model that fuses AIS, satellite SAR observations, and coastal radar to detect dark IUU fishing operations, predict transshipment rendezvous events, and generate optimal interception transit routes for Coast Guard enforcement cutters within 6 hours of confirmation.

Output

1. Complete, reproducible training and inference source code (train.py, predict.py)
2. Exported model weights/checkpoints (.pt or .onnx)
3. Predictions CSV mapping vessel detections to classification and length estimates

Data

File Structure:

PS3_public/

├── train/

| ├── scene_001/

| | ├── VH_dB.tif

| | └── VV_dB.tif

```

| | | └─ bathymetry.tif
| | | └─ owiWindSpeed.tif
| | | └─ ...
| | └─ ...
| └─ test/
|   └─ scene_555/
|     | └─ VH_dB.tif
|     | └─ ...
|     └─ ...
└─ train_labels.csv
└─ sample_submission.csv

```

Data Description:

Scene Structure (per scene folder):

File	Description	Resolution
VH_dB.tif	VH-polarization SAR image	10m pixels
VV_dB.tif	VV-polarization SAR image	10m pixels
bathymetry.tif	Water depth (meters)	500m pixels
owiWindSpeed.tif	Wind speed (m/s)	500m pixels
owiWindDirection.tif	Wind direction (degrees)	500m pixels
owiMask.tif	Land/ice mask	500m pixels

train_labels.csv:

Column	Description	Type
detect_id	Unique detection ID	int
scene_id	Scene identifier	string
detect_scene_row	Pixel row coordinate	int
detect_scene_column	Pixel column coordinate	int
is_vessel	Is it a vessel? (True/False)	bool
is_fishing	Is it fishing? (True/False)	bool
vessel_length_m	Vessel length (meters)	float
distance_from_shore_km	Distance to shoreline	float

Note: Labels are only in train_labels.csv

Submission Format

File: teamname_PS3.csv

1. detect_scene_row,detect_scene_column,scene_id,is_vessel,is_fishing,vessel_length_m
2. 1234,5678,scene_555,True,True,45.2
3. 2345,6789,scene_555,True,False,120.5
4. 3456,7890,scene_556,False,False,0.0

Requirements:

1. Must have exactly 6 columns as shown above
2. detect_scene_row, detect_scene_column: Integer pixel coordinates
3. scene_id: Must match test scene folder names

4. is_vessel, is_fishing: Boolean (True/False)
5. vessel_length_m: Float ≥ 0.0 (use 0.0 for non-vessels)
6. One row per detected object (can have multiple detections per scene)
7. File must be named: teamname_PS3.csv

Evaluation Metric

The evaluation uses 5 sub-metrics combined into an aggregate **MR Score**:

Metric	Description	Direction
F1D	Maritime object detection F1 score	Higher is better
F1S	Close-to-shore detection F1 score	Higher is better
F1V	Vessel classification F1 score	Higher is better
F1F	Fishing classification F1 score	Higher is better
PEL	Length estimation accuracy	Higher is better

Final Score (MR) = $F1D \times (1 + (F1S + F1V + F1F + PEL) / 5)$ (Higher MR score wins)

Problem Statement 04: Ship Detection from Satellite Imagery

Background

Maritime Domain Awareness (MDA) requires continuous monitoring of vessel traffic across vast ocean areas. Satellite imagery provides wide-area coverage, but manual ship detection is time-consuming and error-prone.

Problem

Ship detection in optical satellite imagery is challenging due to varying lighting conditions, cloud cover, sea state, and the small size of vessels relative to image resolution. Automated detection systems must balance sensitivity (detecting all ships) with specificity (avoiding false positives from waves, clouds, and other artifacts).

Aim

Develop a computer vision model that detects ships in optical satellite images and predicts their presence probability and count.

Output

1. Complete, reproducible training and inference source code (train.py, predict.py)
2. Exported model weights/checkpoints (.pt or .onnx)
3. Predictions CSV mapping image_id to ship presence probability and count

Data

File Structure:

PS4_public/

```
|— train/
|   |— img_00001.jpg
|   |— img_00002.jpg
|   |— ...
|— test/
|   |— img_05001.jpg
```

```
| |— img_05002.jpg
| |— ...
|— train_labels.csv
|— sample_submission.csv
```

Data Description:

Images:

1. **Format:** JPG (RGB)
2. **Size:** 768 × 768 pixels
3. **Source:** Sentinel-2 satellite imagery
4. **Coverage:** Global shipping lanes, ports, coastal areas

train_labels.csv:

Column	Description	Type
image_id	Unique image identifier	int
image_path	Relative path to image	string
ship_present	Is there a ship? (0/1)	int
ship_count	Number of ships	int
ship_confidence	Detection confidence (0-1)	float

Note: Labels are only in train_labels.csv

Submission Format

File: teamname_PS4.csv

1. image_id,ship_present_prob,ship_count
2. 5001,0.95,3
3. 5002,0.12,0

4. 5003,0.78,1

Requirements:

1. Must have exactly 3 columns: image_id, ship_present_prob, ship_count
2. ship_present_prob: Float between 0.0 and 1.0
3. ship_count: Integer ≥ 0
4. No missing values (NaN)
5. Same number of rows as test set (1,250 rows)
6. File must be named: teamname_PS4.csv

Evaluation Metric

Your submission will be scored on:

Metric	Description	Direction	Weight
AUC-ROC	Ship detection accuracy	Higher is better	60%
MAE	Ship count estimation error	Lower is better	40%

Final Score = AUC - (MAE / 10) (Higher score wins)

Problem Statement 05: Acoustic Species Classification

Background

Passive acoustic monitoring (PAM) is used to track biodiversity in marine and coastal ecosystems. Manual annotation of audio recordings is labor-intensive and requires expert knowledge.

Problem

Marine and coastal environments host diverse species with overlapping vocalizations. Automated classification systems must distinguish between multiple species in noisy recordings, handle varying recording quality, and account for temporal patterns in vocal behavior.

Aim

Develop a multi-label audio classification model that identifies species from underwater audio recordings.

Output

1. Complete, reproducible training and inference source code (train.py, predict.py)
2. Exported model weights/checkpoints (.pt or .onnx)
3. Predictions CSV mapping audio_id to species presence probabilities

Data

File Structure:

PS5_public/

```
|— train/
|   |— audio_001.wav
|   |— audio_002.wav
|   |— ...
|— test/
|   |— audio_2001.wav
```

| |— audio_2002.wav
| |— ...
|— train_labels.csv
|— sample_submission.csv

Data Description:

Audio Files:

1. **Format:** WAV (mono)
2. **Duration:** 5 seconds per clip
3. **Sample Rate:** 32,000 Hz
4. **Content:** Underwater acoustic recordings (whales, dolphins, fish, ambient noise)

train_labels.csv:

Column	Description	Type
audio_id	Unique audio identifier	int
audio_path	Relative path to audio	string
species_1	Species 1 present? (0/1)	int
species_2	Species 2 present? (0/1)	int
species_3	Species 3 present? (0/1)	int
...	(up to 10 species)	...
ambient_noise	Only ambient noise? (0/1)	int

Note: Labels are only in train_labels.csv

Submission Format

File: teamname_PS5.csv

1. audio_id,species_1_prob,species_2_prob,species_3_prob,...,species_10_prob,ambient_noise_prob
2. 2001,0.95,0.12,0.03,...,0.01,0.02
3. 2002,0.05,0.88,0.15,...,0.03,0.01

Requirements:

1. Must have 12 columns: audio_id + 10 species + ambient_noise
2. All probabilities: Float between 0.0 and 1.0
3. No missing values (NaN)
4. Same number of rows as test set (500 rows)
5. File must be named: teamname_PS5.csv

Evaluation Metric

Your submission will be scored on:

Metric	Description	Direction	Weight
mAP	Mean Average Precision (multi-label)	Higher is better	100%

Final Score = mAP (mean average precision across all species) (Higher score wins)

Dataset Download

Download the complete dataset package (all 5 problem statements) from:
<https://drive.google.com/file/d/1d641heDkInW7tfT1baZ1DxqMoammal6z/view?usp=sharing>

Problems above are real-world maritime challenges where your ML models can directly protect oceans, prevent environmental disasters, and save lives.

Build AI that matters beyond just accuracy metrics.